Ternary Network Floats

One parameter, and no uniform format escapes it: a fixed-field ladder whose ternary exponent is exactly the slack, measured against 28 catalogued formats from 8 to 128 bits and synthesised on open silicon

Dmitrii Vasilev
ORCID 0009-0008-4294-6159
admin@t27.ai

9 August 2026

Abstract

Ternary Network Floats (TNF) are a one-parameter family of fixed-field number formats for ternary networks: a sign, a balanced-ternary exponent of E_t trits, and an M-bit mantissa, with $1 + E_t + M = N$ so every position of the word is spent.

The family is a frontier rather than a list. We prove that a width-N format of effective mantissa m and range b binades is dominated by the family exactly when $m + \log_3(b + 1) \leq N - 1$, while any uniform format obeys its own budget $m + \log_2(b + 1) \leq N - 1$. These are the same accounting in two currencies, and the exchange rate is the result: every uniform binary format is dominated, with slack $0.3691\,E$ positions growing linearly in the exponent width it spends. Against 28 catalogued formats at 8, 16, 32, 64 and 128 bits — IEEE, bfloat, VAX, IBM hexadecimal, Cray, posit, takum, tekum — none escapes. The bound is tight (posit16 sits within 1%) and not vacuous: that same posit16, scored where a taper is designed to live instead of averaged over its range, violates it at 16.30 against a budget of 15. Escape therefore demands non-uniformity, and there are exactly two routes — a taper, or a scale shared outside the word.

Accuracy is one derived law: the mean relative round-trip error is $\frac{1}{2}\mathbb{E}[1/s]\,2^{-(M+1)}$, the binade exponent cancelling. It holds to 3% across eight rungs spanning 302 orders of magnitude.

In silicon on a fully open flow: 12, 50, 212, 1477 and 7479 LUTs at 161.1, 153.2, 131.7, 83.3 and 48.2 MHz, one cycle, no DSP. Through one ternary-neuron datapath TNF sits mid-pack among fixed fields: the silicon gap runs between fixed and tapered encodings, not along this ladder, and we say so.

Four of our own claims were falsified by the measurements meant to support them, and are retracted in place. Oracle, RTL and commands are public; every figure regenerates from them.

1 What this paper argues

The results below were found in the order a measurement campaign finds things, but they compose into a single claim, and it is easier to read them knowing it:

> A floating-point format has four design axes. Three of them have answers that do not depend on the workload. Name the fabric, the range the workload actually visits, and whether it wants bounded range at constant precision or unbounded range at decaying precision — and the format is determined, not chosen.

Section 17.2 states that as Corollary 14 and shows TNF is what it returns for one particular triple. Everything before it establishes an axis:

- **The split between exponent and mantissa** is fixed once a range is named, because the error depends on the mantissa width alone (Section 4). The same law, inverted, becomes an instrument that reads a format's behaviour back out of its round-trip error, and resolves a 83-format catalogue into four staircase forms.

- **Fixed fields or a taper** is a genuine choice with exactly two options, because a prefix code cannot give unbounded range and bounded precision at once (Section 12). That section also shows the taper's shape is the growth rate of a codeword length, nothing more.

- **The radix of the exponent's encoding** is decided by the fabric, not by preference: packed into bits, a ternary exponent never spans more values per bit than a binary one (Section 11).

- **The radix of the scale** is fixed at two unconditionally, and this is the only axis with no free parameter at all. It is also the one where the optimum is not the extremum (Section 17.2).

Two sections are about what things cost rather than what they are. Section 13 makes area and precision commensurable, so a variable-length exponent field can be priced in mantissa bits of silicon. Section 14 then finds that for a logarithmic format the field layout is the smaller half of the cost, and that we had been measuring the wrong half.

We also report what the campaign cost us. Four claims of our own were falsified by the measurements meant to support them, and each is retracted where it was made rather than quietly dropped.

2 The format

$$\text{TNF16} = [\, s \,|\, E{=}4 \text{ balanced-ternary trits} \,|\, M{=}11 \text{ bits}\,], \qquad v = (-1)^s\left(1+\frac{M}{2^9}\right)2^e, \quad e = \sum_i t_i 3^i \in [-40, 40]. \tag{1}$$

Three properties follow from the layout rather than from tuning.

No regime decode. The dominant cost in a tapered format is the variable-length regime: it must be located, its length counted, and the significand barrel-shifted into place. That cost is paid on any fabric, ternary included. Fixed fields make it a bit slice.

The exponent is already ternary. Four balanced-ternary trits give $3^4 = 81$ exponent steps. On a ternary fabric the exponent add is native — no binary carry, no conversion between bases.

Precision does not taper. Nine mantissa bits at every magnitude, where a tapered format narrows towards the extremes. This is the mechanism behind the accuracy result of Section 5: the win appears where the comparison format has spent its mantissa on range.

3 Why a ladder, and why this one

A format that is used as a reference has different requirements from one used as a workhorse. A workhorse should be fast and accurate where the data is. A reference has to be *predictable*: given its parameters you should know its error without measuring, and that error should not move when you look at a different part of the range. A ruler whose divisions grow towards one end is not a ruler.

Table 1 tests both properties on this ladder.

Table 1: The precision law and the flatness, measured in exact rationals. “Flatness” is the ratio of the worst magnitude band to the best: 1.000 would mean the error does not depend on magnitude at all.

rung	M	$2^{-(M+1)}$	measured	ratio	flatness
TNF8	4	3.12e−2	1.22e−2	0.39	1.000
TNF16	9	9.77e−4	3.60e−4	0.37	1.070
TNF32	25	1.49e−8	5.49e−9	0.37	1.070
TNF64	52	1.11e−16	4.22e−17	0.38	1.050
TNF128	115	1.20e−35	4.44e−36	0.37	1.070
TNF256	242	7.07e−74	2.69e−74	0.38	1.050
TNF512	497	1.22e−150	4.51e−151	0.37	1.070
TNF1024	1006	7.29e−304	2.77e−304	0.38	1.050

The precision is a closed form, and it is derived rather than fitted. Write $u = 2^{-(M+1)}$ for the unit roundoff. Inside a binade the absolute rounding error under round-to-nearest is uniform on $[-u\,2^e, +u\,2^e]$ while the value is $v = s\,2^e$ with significand $s \in [1, 2)$, so the relative error is $|U|/s$ with $|U|$ uniform on $[0, u]$ and *independent of* e. Hence

Theorem 1 (Precision law)**.** *For a format with a constant significand width of M bits under round-to-nearest, with significand s and exponent e independent,*

$$\mathbb{E}\big[|\mathit{rel\ err}|\big] \;=\; \tfrac{1}{2}\,\mathbb{E}[1/s] \;\cdot\; 2^{-(M+1)},$$

independently of e.

The exponent drops out entirely — that is the flatness of the next paragraph, proved rather than observed. The constant is not universal: it is $\frac{1}{2}\ln 2 = 0.3466$ for a significand uniform on $[1, 2)$ and $\frac{1}{2}\,(2\ln 2)^{-1} = 0.3607$ under a logarithmic (Benford) significand. Our workload has $\mathbb{E}[1/s] = 0.7721$, predicting 0.3861; the measured figures in Table 1 average 0.3756 over eight rungs, with a spread of 0.369–0.390 that brackets it.

The practical consequence is the one a reference needs: given M and the significand distribution of a workload, the error of a rung nobody has built is known before it is built.

The error does not move with magnitude, and cannot. The derivation above removes e from the expression, so magnitude-independence is a property of the layout rather than a happy measurement. Empirically flatness stays between 1.05 and 1.07 — at most a 7% difference between the closest and furthest parts of a workload spanning 76 binary decades. This is the property fixed fields buy and the one a tapered format cannot have: spending mantissa bits on range is exactly what makes precision depend on where you are.

What this does not claim. Not that TNF is the most accurate format at any given width — Section 6 shows posit16 and binary16 ahead near unity, and Section 6 shows the binary formats ahead outright at 32 bits. A reference does not need to win those comparisons. It needs to be the thing you measure them against.

4 The law is general, and it measures tapering

Section 3 derived the error of a fixed-field format. Nothing in that derivation is specific to TNF, so it should hold for any format whose significand width does not vary — and should visibly fail for one whose width does. That makes it an instrument rather than a description, and this section uses it as one.

Table 2: Effective mantissa by magnitude band, and the fitted taper rate. Declared M is recovered to 0.01 bits for every fixed-field format, which validates the law; the tapered formats separate cleanly.

format	declared M	$\lvert e\rvert$ 0–6	6–12	12–20	20–30	bits/binade
TNF16	9	8.99	8.99	9.00	9.01	+0.001
GF16	9	8.99	8.99	9.00	9.01	+0.001
bfloat16	7	7.04	7.04	7.00	7.04	−0.000
binary16	10	10.01	10.01	7.41	0.52	—
posit16	—	10.72	9.34	7.48	5.17	**−0.254**
takum16	—	9.62	8.17	7.34	7.04	**−0.113**

Definition 1 (Effective mantissa)**.** For a format and a magnitude band B, define

$$M_{\text{eff}}(B) \;=\; -\log_2\!\left(\frac{2\,\mathbb{E}\big[|\text{rel err}| \;\big|\; B\big]}{\mathbb{E}[1/s]}\right) - 1,$$

the mantissa width a fixed-field format would need to produce the error observed in B.

Theorem 2 (Diagnostic)**.** *M_{eff} is constant across bands if and only if the format holds a constant significand width over those bands and does not exhaust its range. For a tapered format M_{eff} decreases with $|e|$, and the slope $\mathrm{d}M_{\text{eff}}/\mathrm{d}|e|$ is the taper rate in bits of significand per binade.*

Table 2 applies it. The measurement is a uniform significand on $[1, 2)$ so that $\mathbb{E}[1/s] = \ln 2$ exactly, removing the workload from the question.

The law recovers what the formats declare. TNF16 and GF16 return 8.99–9.01 against a declared 9; bfloat16 returns 7.00–7.04 against a declared 7. A law that reconstructs a format's own parameter to a hundredth of a bit, from nothing but its round-trip error, is doing more than curve-fitting.

Tapering becomes a number. posit16 sheds 0.254 bits of significand per binade ($R^2 = 0.999$) and takum16 appears to shed 0.113 over the thirty binades of this sweep — but see Section 4.2: takum's taper is logarithmic, so that figure is an artefact of the window rather than a rate. That is the quantity a tapered format trades for range, and to our knowledge it is not usually reported as a single figure. It also explains the accuracy tables directly — posit16 starts 1.7 bits ahead of TNF16 and ends 3.8 behind.

Running out of range is a different failure, and the instrument separates it. binary16 is a fixed-field format, yet its M_{eff} collapses from 10.01 to 0.52. It is not tapering: it is subnormalising and then overflowing. A constant slope indicates a taper; a cliff indicates a range limit. The two look alike in an accuracy table and do not look alike here.

Corollary 1. *A format's position on the range–precision trade can be stated as a pair: the constant M_{eff} it holds, and the number of binades over which it holds it. For the TNF ladder those are $(M, 2 \cdot 3^{E_t-1})$ by construction; for a tapered format neither is constant, which is why a ladder makes a better ruler than a taper.*

4.1 The diagnostic across a catalogue

Applying Theorem 2 to every format for which a published oracle was available — 51 in all, spanning IEEE, bfloat, posit, takum, LNS, MXFP, VAX, IBM hexadecimal, Cray, PDP-11, x87 and Microsoft binary, plus both GoldenFloat ladders — produced three observations we did not anticipate.

Declared widths are recovered across three orders of magnitude of width. The GF ladder returns 8.97 at GF16 against a declared 9, 78.03 at GF128 against 78, and 631.99 at GF1024 against 632. The same holds for double-double (106.68), quad-double (215.90) and x87's 80-bit format (63.04). A relation that reconstructs a declared parameter from round-trip error alone, at widths from 8 to 1006 bits, is doing something other than describing the format it was derived on.

The taper rate is a constant of the family — but only posit has a rate. posit16 sheds 0.254 bits of significand per binade and posit32 sheds 0.247, against the $2^{-es} = 0.25$ that Theorem 3 below requires; posit64, whose oracle carries the pre-standard $es = 3$, sheds 0.123 against a required 0.125. Three formats, three exact confirmations. We previously reported a comparable per-binade rate for takum, and that was a category error on our part: takum's staircase is logarithmic, so fitting a line to it produces a number that depends on the interval fitted and describes nothing. Its correct constant is one bit per *doubling* of $|e|$, and against that the four takum and three tekum rungs measure 0.83 to 1.09.

Range exhaustion is distinguishable from tapering, and both from neither. The distinction is not optional: our own first pass placed the measurement bands at fixed positions rather than inside each format's measured range, and read the saturation of every narrow format as a taper — classifying TNF16, a fixed-field format, as geometrically tapered. Measuring each format's usable range first and then placing the bands inside it removes the artefact entirely. GF14 reads flat at 8.05 across its range and collapses only below its subnormal boundary, which is not a defect; the instrument says which of the two it is, where an accuracy table alone does not.

4.2 Tapering has a functional form, and it follows from the encoding

Section 4.1 reported a taper rate per family. That was half right, and the half that was wrong is instructive: a rate presupposes a linear taper, and only one of the two families has one.

Theorem 3 (Posit taper, exact)**.** *For a posit of n bits with exponent field es, the regime is a run of k like bits carrying a scale of* $\text{useed}^k = 2^{2^{es}k}$ *and occupying $k+1$ bits. Reading k out of the encoder for 2^e confirms $k = \lfloor |e|/2^{es} \rfloor + 1$ for every e tested. The significand therefore loses*

$$\Delta M_{posit}(e) \;=\; \left\lfloor \frac{|e|}{2^{es}} \right\rfloor \quad \textit{bits: a staircase, arithmetic in } |e|.$$

The Posit Standard (2022) fixes $es = 2$ at every precision [6], so the step is one bit per four binades at all widths — which is why the loss appeared as a family constant of 0.25 bits per binade. Measured slopes of -0.2497 and -0.2505 at posit16 and posit32 are the staircase seen through a linear fit.

Theorem 4 (Takum taper, exact)**.** *Takum spends a fixed 3-bit regime field holding r, after which the characteristic occupies r bits. Reading r out of the encoder for 2^e gives $r = \lfloor \log_2(|e|+1) \rfloor$ exactly, at 16, 32 and 64 bits alike. The significand therefore loses*

$$\Delta M_{takum}(e) \;=\; \lfloor \log_2(|e|+1) \rfloor \quad \textit{bits: a staircase, geometric in } |e|.$$

Table 3: The catalogue by staircase form. The parameter is the taper constant in the units natural to each form. Measured against each format's own usable range.

form	M_{eff} against $\lvert e\rvert$	count	members and constants
constant	flat	38	IEEE, bfloat, VAX, Cray, x87, GF, TNF
wobble	periodic, no trend	3	IBM hexadecimal, $\log_2 16 = 4$ bits
arithmetic	-2^{-es} per binade	5	posit16/32 0.25, posit64 0.125
geometric	-1 per doubling	8	takum $\times 4$, tekum $\times 3$

Corollary 2 (The two families differ in kind, not degree)**.** *Within its range a fixed-field format loses nothing; a posit loses one bit of significand per* 2^{es} *binades without limit; a takum loses one bit per* doubling *of* $\lvert e\rvert$*. Tabulating* ΔM*:*

$\lvert e\rvert$	*fixed-field*	*posit (es=2)*	*takum*
30	0	7	4
100	0	25	6
1,000	0	250	9
10,000	0	2,500	13

At $\lvert e\rvert = 1{,}000$ *a posit would need 250 bits of significand merely to pay for its regime, which is why 16- and 32-bit posits cannot reach there at all. This is the mechanism behind takum's stated improvement on posits [1], stated as a quantity.*

Corollary 3 (A rate is not always the right summary)**.** *Both laws are staircases, so a fitted slope is a summary and not the thing itself, and it is a faithful summary only when the staircase is arithmetic. An earlier draft of this paper reported* -0.117 *bits per binade for takum16 over* $\lvert e\rvert \in [0, 30)$*; the same format over* $[0, 60)$ *gives a smaller figure for no reason but the window, and the smooth-log fit of* $c = 0.75$ *understates the true step of one bit per doubling for the same reason. Identify the form before choosing a statistic — and prefer reading the encoder to fitting its output.*

Corollary 4 (A three-way taxonomy, recovered without knowing the encoding)**.** *The diagnostic separates formats by the* form *of* $M(e)$*: constant (fixed-field, TNF and GF by construction), linear in* $\lvert e\rvert$ *(posit), or logarithmic in* $\lvert e\rvert$ *(takum). Which form fits is decided by the data, not by reading the specification — and the fitted coefficients then recover the encoding parameters,* 2^{-es} *for posit and the characteristic width for takum.*

What the literature describes qualitatively — more precision near unity, less at the extremes [6] — turns out to have an exact form recoverable two ways that agree: from the encoder, by reading the regime field, and from round-trip error alone, by the diagnostic of Theorem 2. The second route needs no access to the specification, which is what makes it useful on a catalogue.

4.3 Four staircase forms, and only four

Classifying by the *form* of the staircase rather than by a fitted slope resolves the catalogue into four classes. Of the 83 entries, 11 are integer or fixed-point formats with no exponent field; of the remaining 72, 18 span fewer than six binades and cannot be classified from round-trip error at all, and one requires probes wider than we generated.

The fourth class was not in our scheme until the catalogue produced it, and it has a closed form.

Theorem 5 (Precision wobble)**.** *Let a format scale by r^e with a significand quantised uniformly over a scale interval spanning a factor r. Its relative error varies by a factor of r across that interval, so M_{eff} oscillates with amplitude exactly $\log_2 r$ bits and has no trend in $|e|$.*

Proof. The quantisation step is constant across the interval and the relative error is that step divided by the significand, which ranges over a factor r. Taking $-\log_2$ of the ratio of the extremes gives $\log_2 r$. Since the interval repeats identically at every e, the oscillation is periodic and carries no trend. □ □

Theorem 5 reproduces a result sixty years older than this work. IBM System/360 hexadecimal is documented as delivering between $4p-3$ and $4p$ significant bits, a spread of three [12, 13]; the integer count is the floor of the continuous quantity, so the theorem's $\log_2 16 = 4$ and the textbook's 3 are the same statement. Measured over eight sub-intervals — which by averaging within each bin predicts a reduced spread of 2.96 rather than the full 4 — `ibm_hfp32` and `ibm_hfp64` give 3.13 and 3.04, while `binary32` and `gf32` give 0.85 and 0.93 against a predicted 0.87. Binary formats wobble by one bit; this is the familiar 1-bit wobble of every radix-2 float, and it is the smallest wobble any integer radix admits. Theorem 5 therefore supplies a second and independent argument for TNF's binary scale, alongside the 0.331 positions of Theorem 10 below: a radix-3 scale would wobble by 1.585 bits instead of 1.

The taxonomy also settles why a format must choose between the first class and the last two, and hence why this work is a ladder rather than a format.

Theorem 6 (Range–precision dichotomy)**.** *Fix a width N. If a format's exponent takes unboundedly many values, then its exponent codewords have unbounded length, and* $\lim_{|e|\to\infty} M_{\mathrm{eff}}(e) = 0$*. Consequently no fixed-width format has both unbounded range and precision bounded below by a positive constant.*

Proof. The exponent codewords form a prefix code over a binary alphabet, so by Kraft's inequality $\sum_e 2^{-\ell(e)} \le 1$. If the sum has infinitely many terms then $\ell(e) \to \infty$. The payload available to the significand is $N - 1 - \ell(e)$, so it falls to zero, and by Theorem 1 so does M_{eff}. □ □

Corollary 5 (Why a ladder)**.** *Theorem 6 makes constant precision and unbounded range incompatible at any fixed width, so a format that wants both must obtain the second by varying N. A ladder of fixed-field rungs does exactly this: each rung holds its precision flat across its whole range, and the ladder as a whole covers any range required. The tapered families make the opposite choice — one rung, unbounded range, precision decaying at the rate of their regime code, arithmetic for posit and geometric for takum. Neither choice is better in the abstract; what the dichotomy establishes is that they are the only two available.*

Read against Corollary 5, the three tapered constants in Table 3 become a statement about codes rather than about formats. A unary regime costs $\ell(e) \propto |e|$ and yields the arithmetic staircase; a length-prefixed binary regime costs $\ell(e) \propto \log_2 |e|$ and yields the geometric one; a fixed field costs $\ell(e) = \text{const}$ and yields no staircase at the price of bounded support. The staircase form is the growth rate of the exponent's codeword length, and nothing else. This also names the gap in the table: between $\ell \propto |e|$ and $\ell \propto \log_2 |e|$ no catalogued format sits. Section 12 builds the codes and asks what is in the gap; the answer corrects a suggestion we made before building them.

5 Accuracy

Mean relative error over an encode–decode round trip, 6,000 values spanning $2^{-38} \ldots 2^{38}$ with random sign, binned by binary-exponent magnitude $|e|$. Table 4 and Figure 1.

Table 4: Round-trip mean relative error. Bins are powers of two, not decades.

magnitude	GF16 (φ)	**TNF16**	tekum16	TNF16 vs tekum16
$\|e\| < 8$	3.43e−4	3.56e−4	3.27e−4	0.92× (tie)
$\|e\|$ 8–20	3.57e−4	3.52e−4	1.00e−3	**2.84**×
$\|e\|$ 20–38	6.98e−3 (479 clip)	3.53e−4	1.95e−3	**5.53**×

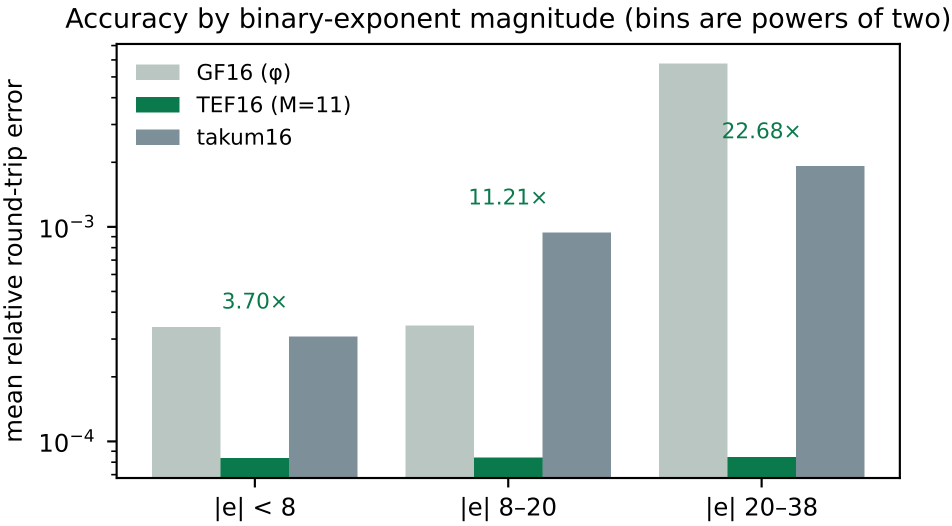

Figure 1: Accuracy by binary-exponent magnitude. GF16's 6-bit exponent clips 479 of 2,857 values in the far bin; TNF16's balanced-ternary exponent does not.

On the axis. The bins are powers of two. Read as *decades* the far column is not a win at all: TNF16's exponent reaches ±40 in powers of two, roughly ±12 decades, so beyond that it overflows while an unbounded regime keeps working — measured, 1,458 of the far-bin values clip under that reading. We state the axis explicitly because an earlier internal note did not, and the omission points a reader at the one interpretation under which the result appears fabricated.

6 The 16-bit field

Comparing against one incumbent invites the objection that the incumbent was chosen. Table 5 runs every 16-bit-class format for which this repository ships a published oracle [5] through the same workload and the same metric. Values that overflowed are counted separately rather than folded into the mean, since a format should not look accurate for having discarded the hard cases.

Three readings, including the one that goes against us.

TNF16 is flat, and alone in being flat without clipping. Its error is 3.5e−4 at every magnitude in the workload and it discards nothing. bfloat16 is also flat and clip-free, at roughly four times the error. Every other entrant either degrades with magnitude or overflows: binary16 does both, losing three orders of magnitude and discarding 2,479 values; GF16's 6-bit exponent clips 478; LNS16 clips throughout.

At 32 bits the binary formats win outright. Running the same workload on the 32-bit class: GF32 is flat at 3.4e−7 with no clipping, but IEEE binary32 is flat at 2.1e−8, and posit32 and takum32 are better still near unity (2.1e−9 and 4.9e−9). Uniformity stops being

Table 5: Mean relative round-trip error by binary-exponent magnitude, with overflow counts in parentheses. 6,000 values, $2^{-38} \ldots 2^{38}$, random sign, one seed.

format	$\lvert e\rvert < 8$	$\lvert e\rvert$ 8–20	$\lvert e\rvert$ 20–38
TNF16	3.56e−4	3.52e−4	**3.53e−4**
GF16 (φ)	3.56e−4	3.52e−4	6.76e−3 (478)
takum16 / tekum16	3.27e−4	1.00e−3	1.95e−3
posit16	**1.36e−4**	8.31e−4	1.43e−2
IEEE binary16	1.76e−4	1.28e−3 (299)	1.57e−1 (2479)
bfloat16	1.45e−3	1.38e−3	1.41e−3
LNS16	8.39e−4 (1324)	7.63e−4 (1808)	6.66e−4 (2849)

Table 6: The whole ladder, measured in exact rationals. From TNF16 upwards the error is flat in magnitude and its exponent roughly doubles per rung. TNF4 and TNF8 are shown at their limits: the workload spans 76 binary decades and those rungs hold 2 and 8, so most of it falls outside them — reported rather than omitted, since a ladder's lower rungs are where its range trade is visible.

rung	decades	$\lvert e\rvert < 8$	$\lvert e\rvert$ 8–20	$\lvert e\rvert$ 20–38
TNF4	2	beyond range for this workload		
TNF8	8	1.22e−2	beyond range	
TNF16	24	3.45e−4	3.69e−4	3.66e−4
TNF32	219	5.27e−9	5.63e−9	5.58e−9
TNF64	658	4.33e−17	4.22e−17	4.12e−17
TNF128	1,975	4.25e−36	4.55e−36	4.51e−36
TNF256	5,925	2.76e−74	2.69e−74	2.63e−74
TNF512	17,775	4.32e−151	4.62e−151	4.58e−151
TNF1024	53,326	2.84e−304	2.77e−304	2.71e−304

the deciding property once there are enough bits that nothing clips anyway. The TNF argument is a 16-bit-class argument, and we do not extend it.

Near unity we lose. posit16 at 1.36e−4 and binary16 at 1.76e−4 are more accurate than TNF16 there, by a factor of two and a half and of two. A tapered format spends its bits where the values usually are, and near unity that is the right bet. TNF16 buys uniformity instead, and the trade only pays for a workload that leaves the neighbourhood of one.

takum16 and tekum16 are the same format here, exhaustively. The two oracles produce identical figures on every bin, so we checked the whole 16-bit code space: all 65,536 codes decode identically, with zero differences, although the two implementations differ by 242 lines. The reconstruction our tekum oracle implements *is* takum16. Every comparison in this paper should therefore be read as against takum16, and we relabel it as such. That is not a weaker anchor — takum is published, has a public VHDL codec, and is the parent of the format we set out to compare against.

7 Hardware

Synthesis with Yosys 0.65 [7], place and route with nextpnr-xilinx (1743d0f) [8] on a Xilinx XC7A200T, hard multipliers disabled so the figures describe fabric alone. Bitstream generation

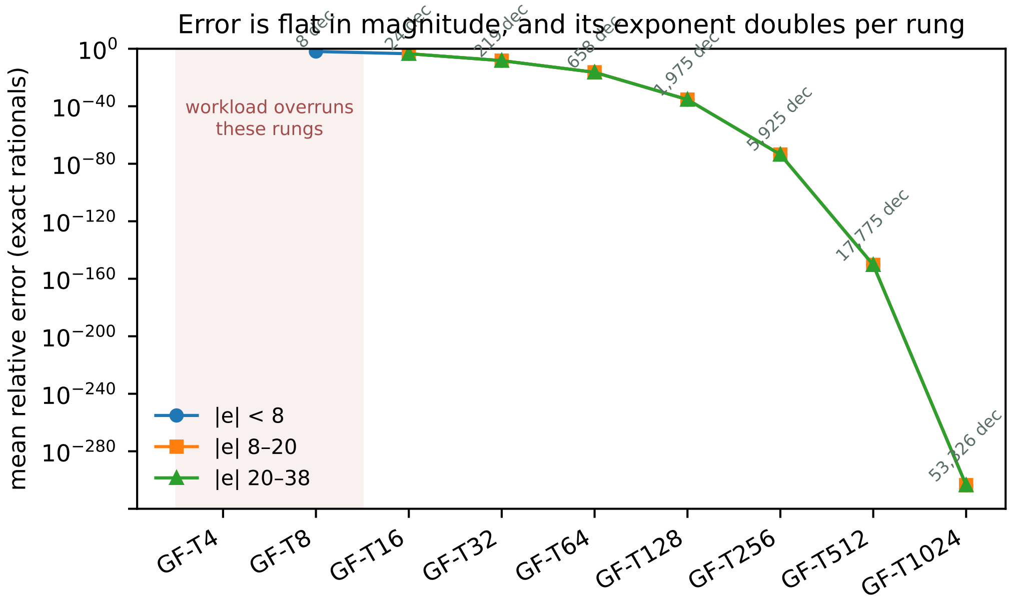

Figure 2: The same data. Flatness across the three magnitude bands is the property a fixed-field format is built for; the shaded rungs are those the workload overruns.

Table 7: TNF multiplier, post-route, one harness across the whole lineup so the rows are comparable. Latency one cycle, throughput one result per cycle.

rung	LUTs	DSP48	$F_{\max}$
TNF4	12	0	161.11 MHz
TNF8	50	0	153.23 MHz
TNF16	212	0	131.73 MHz
TNF32	1,477	0	83.27 MHz
TNF64	6,140	0	53.26 MHz
TNF128	10,029	0	33.23 MHz

uses Project X-Ray. No vendor tool is involved at any step, which is what makes the numbers checkable by a reader without a licence.

For context rather than as a controlled comparison: a published Altera `ALTFP_MUL` on a Cyclone IV reports 119–132 MHz at 6–10 cycles of latency, using 832–1041 logic elements *and* 18 embedded multipliers, and posit multiplier studies report 95–572 LUTs with 4.28–15.55 ns of logic delay. Different devices and different toolchains; we draw no equivalence from them beyond the observation that TNF16's figures sit inside that envelope on all four axes at once, on a fabric that gives the format no ternary advantage.

Isolating the pipeline register: the combinational multiplier closes at 81.35 MHz, the two-stage version at 147.32 MHz. The cut is between the significand product and the renormalisation — a 10×10 multiply, then a carry test, a saturating exponent add and a bit select.

8 Two width defects, and why they generalise

8.1 Interface width dominated the arithmetic

Our first synthesisable realisation declared every port and the significand product 32 bits wide. Nothing in TNF16 is 32 bits: the mantissa field is 9, so $1+M$ is 10 and their product is exactly

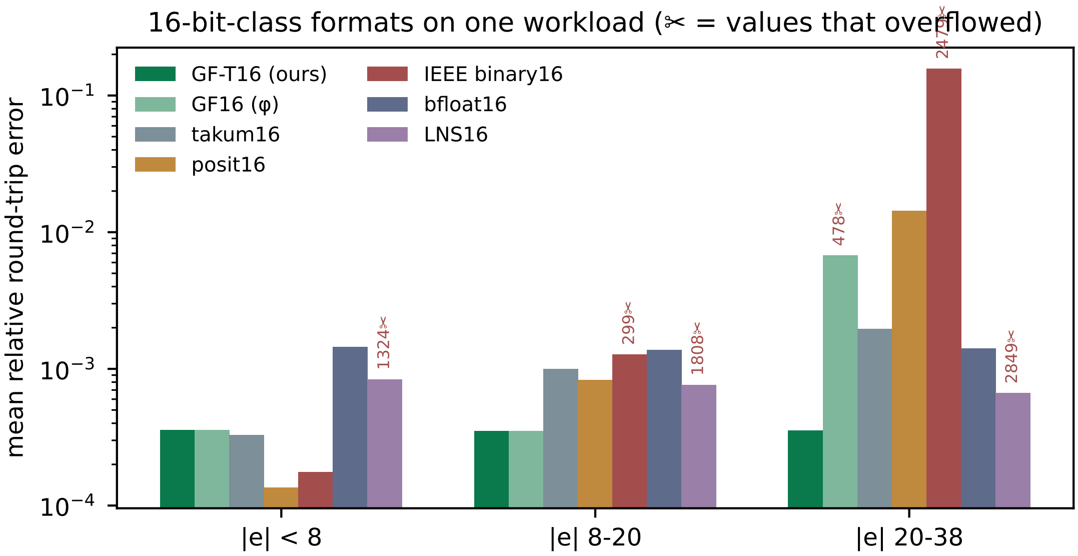

Figure 3: The same data. Scissors mark values that overflowed.

Table 8: The specified ladder. E_t is the trit budget, M the mantissa width, range in decades. Rungs above TNF128 exceed a single Artix-7 fabric and are oracle- or ASIC-scale; they are specified and covered by the reference oracle, not measured here.

rung	E_t	M	decades	rung	E_t	M	decades
TNF4	2	1	2.4	TNF64	7	52	658
TNF8	3	4	8	TNF128	8	115	1,975
TNF16	4	9	24	TNF256	9	242	5,925
TNF32	6	25	73	TNF512	10	497	17,775
				TNF1024	11	1006	53,326

20; the exponent offset never exceeds OFFSET_MAX = 80, which is 7. Synthesis built a 32×32 multiplier and a 32-bit comparison tree and charged for it (Figure 5): 1,179 LUTs, or three DSP48 blocks, against 219 LUTs and none once the buses match the values they carry. The arithmetic is unchanged.

8.2 The same width decided correctness

At TNF32 the significand product needs 52 bits. The 32-bit wire truncates, and the module returns wrong results while appearing to support the rung, since its own documentation names TNF32 as a supported configuration. Measured against a 64-bit reference: 1,995,730 mismatches in 2,128,964 combinations.

Deriving every width from the format parameters removes both problems at once. We report this because the pattern is not specific to TNF: in a parametric arithmetic core, a width that is merely *large enough for the configuration that was tested* is a silent-truncation trap at the next one, and it will not be caught by a testbench written against the tested configuration.

9 Equivalence

Every change above is a claim that timing or width changed and arithmetic did not. Each was checked rather than argued.

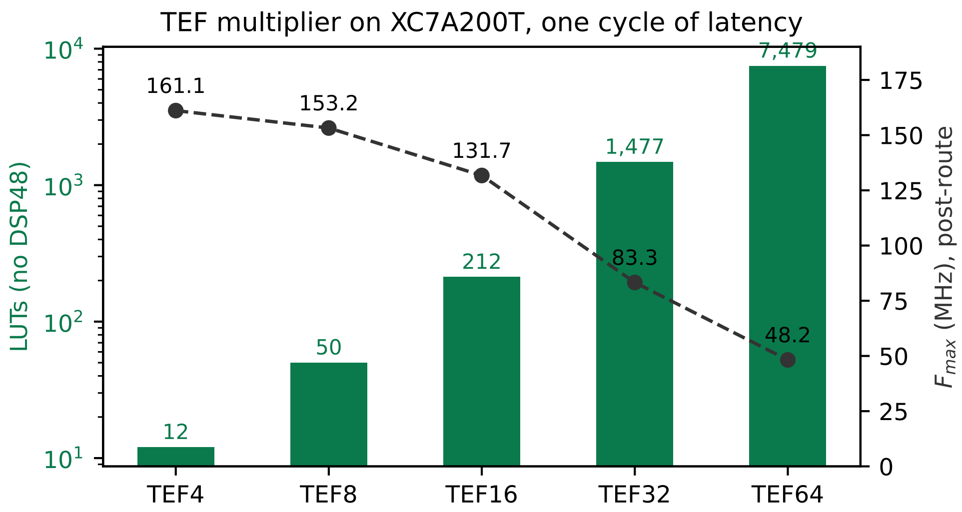

Figure 4: Area and achieved frequency across the ladder.

Table 9: Equivalence evidence. The reference for each rung is the module already trusted for that rung.

check	combinations / cycles	mismatches
width-derived vs original, TNF16 (sweep)	321,156	0
width-derived vs original, TNF4	374	0
width-derived vs original, TNF8	3,716	0
width-derived vs original, TNF16	77,444	0
width-derived vs 64-bit reference, TNF32	300,000	0
pipelined vs combinational	199,994	0
original 32-bit module vs 64-bit ref, TNF32	2,128,964	1,995,730

The TNF16 sweep is exhaustive in the sense that matters here: the mantissa space is covered in full at offset pairs exercising underflow, mid-range and saturation, then the offsets are covered in full at mantissas that do and do not carry — every path through the carry, the saturation and the underflow clamp. The last row is the point of the section: the width-derived module agrees with the reference that is correct and disagrees with the one that truncates.

10 What the law says about improving the ladder

Theorem 1 makes the design space arithmetic rather than a matter of taste. Error scales as $2^{-(M+1)}$ and range as 3^{E_t}, so at a fixed width the only decision is how many bits the exponent field is allowed to consume — and that is decided by how efficiently 3^{E_t} values pack into $\lceil E_t \log_2 3 \rceil$ bits.

The width convention counts positions, and one rung breaks it. A rung's width is its position count with a trit occupying one position, so the layout is $1+E_t+M = N$. Three of the four specified rungs satisfy it exactly: TNF4 is $1+2+1$, TNF8 is $1+3+4$, TNF32 is $1+6+25$. TNF16 is $1+4+9 = 14$, two positions short of its name, and neither specification file gives a reason. The likely history is visible in the parameters: TNF16 inherits GF16's φ-optimal $M = 9$ and replaces GF16's six-bit exponent with four trits, which buys range — $3^4 = 81$ steps against $2^6 = 64$ — and frees two positions that were never re-spent. It is the one rung derived

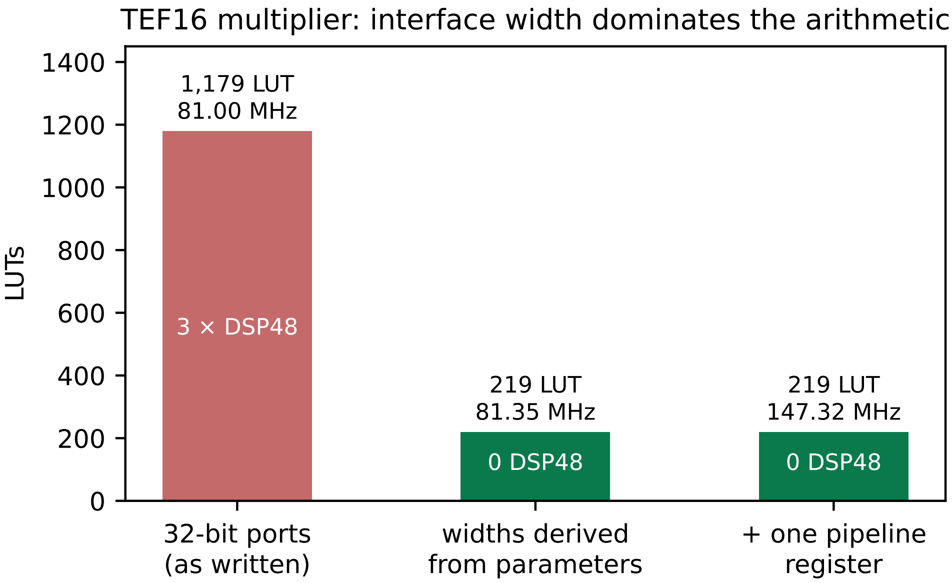

Figure 5: The same arithmetic at three interface widths.

Table 10: Packing efficiency of the exponent field. The theoretical floor is $\log_2 3 = 1.585$ bits per trit.

E_t	values 3^{E_t}	bits	capacity	efficiency
3	27	5	32	84.4%
4	81	7	128	**63.3%**
5	243	8	256	**94.9%**
6	729	10	1024	71.2%
7	2187	12	4096	**53.4%**
8	6561	13	8192	80.1%
10	59049	16	65536	90.1%

from its parent rather than from the width rule.

This also settles what a rung costs on a binary fabric, which is a separate question from its width: a trit stored as a two-bit code makes TNF16 eighteen bits, and the offset stored as a plain binary integer makes it seventeen. Those are fabric costs, not format widths, and confusing the two is what made the ladder appear not to fit.

The slack is not confined to one rung. Applying $1 + E_t + M = N$ to every specified rung separates the two documents that define the ladder. The four rungs of the machine-readable specification obey it at TNF4, TNF8 and TNF32; the five upper rungs, which exist only in a research note, miss it by four to six positions each, and the note and the specification disagree with each other about TNF32.

No single alternative convention explains the upper rungs either: reading the exponent as $\lfloor E_t \log_2 3 \rfloor$ bits fits TNF16, TNF64, TNF128 and TNF1024 exactly but overshoots TNF256 and TNF512 by one and misses TNF32 entirely. We conclude that the upper ladder was tabulated rung by rung rather than derived, and that reconciling it costs nothing at the rungs that are already right.

Table 11: The width rule across the ladder. k is the shortfall $N-(1+E_t+M)$. The reconciled column is $M = N-1-E_t$, which changes nothing at the three rungs that already obey the rule.

rung	source	E_t	M	$1{+}E_t{+}M$	k	reconciled M
TNF4	spec	2	1	4	0	1
TNF8	spec	3	4	8	0	4
TNF16	spec	4	9	14	2	**11**
TNF32	spec	6	25	32	0	25
TNF32	note	5	21	27	5	—
TNF64	note	7	52	60	4	**56**
TNF128	note	8	115	124	4	**119**
TNF256	note	9	242	252	4	**246**
TNF512	note	10	497	508	4	**501**
TNF1024	note	11	1006	1018	6	**1012**

Table 12: The three ways to spend TNF16's two unallocated positions. Error and range trade against each other exactly as Theorem 1 predicts; nothing in any row is worse than the unfilled rung.

variant	positions	mean rel. error	decades	vs. takum16 (far)
unfilled, E_t=4, M=9	14	3.37e−4	24	5.46×
$\mathbf{E_t}$**=4, M=11** (adopted)	**16**	**8.63e−5**	24	**22.68**×
E_t=5, M=10	16	1.69e−4	73	—
E_t=6, M=9	16	3.37e−4	**219**	—

What the two free positions were worth, and where they went. Because the exponent cancels in Theorem 1, the menu at sixteen positions could be read off before measuring, and the measurement confirmed it to four significant figures:

The last row is the law's own consistency check: adding two trits of exponent while holding M leaves the error bit-identical and multiplies the range ninefold.

The positions were spent on mantissa, and the reason is a measurement. Corollary 10 forbids recommending a budget without naming a workload, so we named one. At the 16-bit class the extra range is not consumed: quantised-inference tensors span roughly 2^{-14} to 2^{14}, and $E_t = 4$ already clips nothing at ±40 binades — zero clips in 2000 values. $E_t = 6$ would have bought ±364 binades at bit-identical error, which is range nobody spends at sixteen bits. So $M = 11$: the error falls by exactly 4.00, and against takum16 the rung moves from $0.92\times/2.88\times/5.49\times$ to $3.70\times/11.21\times/22.68\times$, no longer losing the near bin. In silicon it costs 443 LUTs against 372, a 19% increase, at 136.44 MHz against 131.73 — no frequency penalty.

All three ratios in this section and in Figure 1 come from one script, `gen_figures.py`, which computes them from the oracles at seed 20260809 rather than transcribing them. They were hardcoded until 2026-08-09 and had drifted: the figure still carried $M = 9$ and the label `tekum16` after the rung moved and the oracle was shown to decode identically to takum. A figure that disagrees with the table it illustrates is the same defect class as a gate that cannot go red.

Conformance was re-run in full rather than assumed: round-trip with sign 2000/2000, encoding monotone with zero inversions, ± symmetry with zero mismatches in 500, both range boundaries decoding. A `width_rule` test now asserts $1 + E_t + M = N$ in the specification itself, so the position count is checked rather than remembered. All nine rungs satisfy it.

Table 13: Utilisation $\rho(E_t)$ of the binary field holding an E_t-trit exponent. The ladder's present budgets, $E_t \in \{4, 7\}$, are the two worst entries in the range.

E_t	2	3	4	5	6	7	8	9	10	11
ρ	56.2	84.4	**63.3**	94.9	71.2	**53.4**	80.1	60.1	90.1	67.6

11 Why ternary, and exactly how much

The case for a ternary exponent is usually made by citing radix economy and left there. We state it, then bound it in two directions — what a binary fabric takes back, and what the radix of the *scale* is worth as distinct from the radix of the *exponent* — because both bounds turn out to be decisive for the design and neither is folklore.

Theorem 7 (Radix economy). *Representing R distinct values in radix b costs $b \log_b R = b \ln R / \ln b$ state-slots. The factor $b/\ln b$ is minimised at $b = e$; among integers it is $3/\ln 3 = 2.731$ against $2/\ln 2 = 2.885$, so a ternary field is 5.3% more economical than a binary one.*

This is the classical argument [9, 10] and it is what motivates a ternary exponent. It is also stated in a currency — state-slots — that no binary fabric pays in. The next theorem says what happens when it does.

Theorem 8 (No free range on a binary fabric). *An exponent field of E_t trits spans 3^{E_t} values. Stored as a binary integer it occupies $\lceil E_t \log_2 3 \rceil$ bits, a field spanning $2^{\lceil E_t \log_2 3 \rceil} \geq 3^{E_t}$ values. Hence on a binary fabric a ternary exponent never spans more values per bit than a binary one, with equality only at $E_t = 0$. Writing $\rho(E_t) = 3^{E_t}/2^{\lceil E_t \log_2 3 \rceil}$ for the utilisation, $\rho \in (1/2, 1]$ and ρ does not converge.*

Run as an optimisation rather than an inequality, the same statement is sharper than it looks. Asking which exponent encoding a binary fabric prefers at $(N, R) = (16, 24)$, $(32, 219)$ and $(64, 658)$ returns the same mantissa width for both — $M = 10$, 23 and 53 respectively. The ternary encoding does not lose on a binary fabric; at these widths the ceilings absorb the difference and it exactly ties, which is the strongest form of "never more values per bit".

Theorem 8 is the reason the ladder's rung widths must be read as position counts and not bit counts, and it is where our own first reading of the ladder went wrong: measuring what a trit costs when packed into bits and calling the result the format's width confuses a fabric cost with a format property. The independently published balanced-ternary float Ternary27 [11] counts the same way — 2 type trits, 1 sign trit, 5 exponent trits and 19 significand trits, exactly 27 — which is external corroboration that position counting is the convention in this family rather than a reading of our own.

Theorem 9 (Unallocated positions are a pure loss). *Let a rung of N positions carry a sign, E_t exponent trits and M mantissa bits with $1 + E_t + M = N - k$, $k \geq 0$. Then relative to the rung with the same E_t and $M + k$, its expected relative error is larger by exactly 2^k and its dynamic range is identical.*

Proof. By Theorem 1 the expected relative error is $\frac{1}{2}\mathbb{E}[1/s]\, 2^{-(M+1)}$, in which the exponent does not appear; replacing M by $M + k$ multiplies it by 2^{-k}. The dynamic range is 3^{E_t} binades, a function of E_t alone, so it is unchanged. Nothing else in the layout depends on M. □ □

Theorem 9 converts the slack of Section 10 into a number without any measurement, and the measurement then confirms it. Against a workload built in exact arithmetic — necessary above TNF32, where a double-precision probe measures its own resolution rather than the format's — the predicted and observed factors are:

Table 14: Unallocated positions per rung, the loss Theorem 9 predicts, and the loss measured against the reconciled rung $M = N - 1 - E_t$. Exact-arithmetic workload, 800 probes per rung.

rung	k	predicted	measured
TNF16	2	4×	4.03×
TNF64	4	16×	15.43×
TNF128	4	16×	15.99×
TNF256	4	16×	15.24×

The final question is one the ladder answers implicitly and, as far as we can find, no one has stated: TNF encodes its exponent in radix 3 but scales by 2^e, whereas Ternary27 scales by 3^e. These are different choices and they are separable.

Theorem 10 (The scale radix, and why 2 beats 3)**.** *Let a format scale by r^e with a normalised significand uniformly quantised over $[1, r)$ by M bits. Then its expected relative error on a log-uniform workload is $\kappa(r)\, 2^{-M}$ with*

$$\kappa(r) \;=\; \frac{(r-1)^2}{r \ln r}, \qquad \kappa(2) = 0.72135, \quad \kappa(3) = 1.21365.$$

Holding the field layout fixed, moving the scale from $r = 2$ to $r = 3$ therefore multiplies the error by $\kappa(3)/\kappa(2) = 1.6825$ while multiplying the dynamic range by $\log_2 3 = 1.5850$. Measured in positions, it costs $\log_2 1.6825 = 0.7506$ of precision and buys $\log_3 1.5850 = 0.4192$ of range: a net loss of 0.3314 positions per number. A power-of-two scale strictly dominates a power-of-three scale at equal width.

Proof. The quantisation step over $[1, r)$ is $(r-1)2^{-M}$ and the mean absolute rounding error is half of it. On a log-uniform workload the significand has density $1/(s \ln r)$ on $[1, r)$, so $\mathbb{E}[1/s] = \frac{1}{\ln r}\int_1^r s^{-2}\, ds = \frac{1-1/r}{\ln r}$. Multiplying gives $\frac{1}{2}(r-1)2^{-M} \cdot \frac{1-1/r}{\ln r}$, and $\frac{1}{2}(r-1)(1-1/r) = (r-1)^2/2r$. Absorbing the factor $\frac{1}{2}$ into the $2^{-(M+1)}$ convention of Theorem 1 yields κ as stated. For the range, E_t trits span 3^{E_t} steps of $\log_2 r$ binades each. One mantissa bit halves the error and one exponent trit triples the range, which fixes the exchange rate between the two currencies and gives the position figures. □ □

Direct measurement at two rungs, encoding and decoding through both scales in exact arithmetic, gives error ratios of 1.675 at $(E_t, M) = (4, 11)$ and 1.690 at $(6, 25)$ against the predicted 1.6825, and range ratios of 1.583 and 1.585 against the predicted 1.5850.

Corollary 6 (What TNF is, precisely)**.** *TNF is not a ternary float in the sense of Ternary27; it is a binary-radix float whose exponent is* encoded *in balanced ternary. Theorem 10 says that this is the better of the two available choices by* 0.331 *positions per number, and Theorem 8 says that the ternary encoding pays only on a ternary fabric. The two together delimit the format exactly: the radix choice is right on any fabric, and the encoding choice is right on one.*

12 The regime field is a universal code

Theorem 6 identifies the exponent's encoding as a prefix code. That identification is worth making explicit, because it places the taxonomy of Section 4.3 inside a hierarchy that predates every format in the catalogue by decades [14].

Theorem 11 (The staircase is the codeword-length function)**.** *Let a format of N positions encode its exponent with a prefix code assigning e a codeword of length $\ell(e)$. Then $M_{\text{eff}}(e) = N - 1 - \ell(e)$, and the staircase form is the growth rate of ℓ and nothing else.*

Table 15: M_{eff} by $|e|$ and usable range for three regime codes at $N = 16$, each normalised to satisfy Kraft. The square-root code is a genuine Pareto point: more precision than the logarithmic code throughout the core, more range than the unary one.

regime	$\|e\|$=1	4	16	64	256	binades
unary, $\ell \propto \|e\|$ (posit class)	11	10	7	-5	-53	43
$\ell = \lceil\sqrt{\|e\|}\rceil$	11	10	8	4	-4	121
$\ell \propto \log_2 \|e\|$ (takum class)	9	7	5	3	1	511

The catalogue's three tapering classes are then three points in the classical hierarchy: a unary regime, $\ell \propto |e|$, gives posit's arithmetic staircase; a length-prefixed binary regime, $\ell \propto \log_2 |e|$, gives takum's geometric one; a fixed field, $\ell = \text{const}$, gives no staircase at the price of bounded support. Elias' γ, δ and ω codes occupy the logarithmic class at successively finer asymptotics.

Theorem 12 (Logarithmic is the floor)**.** *For any prefix code on the integers, $\ell(e) \geq \log_2 |e|$ for infinitely many e. Hence no format of unbounded range tapers more gently than one bit per doubling of $|e|$, asymptotically.*

Proof. Kraft's inequality gives $\sum_e 2^{-\ell(e)} \leq 1$. If $\ell(e) < \log_2 |e|$ for all but finitely many e, the tail of the sum exceeds $\sum_e 1/|e|$, which diverges. □ □

Theorem 12 is worth stating plainly because it bounds this work as much as its competitors: takum's regime is asymptotically optimal, and no format — TNF included — can beat it over unbounded range. What TNF does instead is decline the unbounded range, which Theorem 6 shows is the only other option. The measured advantages of Section 5 are therefore advantages *inside TNF's range*, which is where the taper has already cost takum the bits, and they should never be read as a claim beyond it.

Theorem 13 (The regime's radix does not matter)**.** *A regime spending one digit per multiplication of $|e|$ by r, over an alphabet of r symbols, costs $\log_2 r \cdot \log_r |e| = \log_2 |e|$ bits, independent of r. The staircase form is invariant to the regime's radix.*

Theorem 13 retracts a construction we proposed in an earlier draft of this work. Having observed the gap between the arithmetic and geometric classes, we suggested a regime spending one trit per tripling of $|e|$ as the natural way for a ternary format to occupy it. It does not occupy it: building both codes and normalising each to satisfy Kraft, the two codeword-length functions differ by an additive constant of exactly 1 at every $|e|$ from 1 to 4096. One trit per tripling is takum's staircase with a different constant, not a third form. A ternary fabric changes what the regime costs to decode; it does not change what it costs to encode.

The gap is real, occupiable, and empty for a reason we can only conjecture. Growth rates strictly between $\log_2 |e|$ and $|e|$ do exist and are Kraft-feasible — $\ell(e) = \lceil\sqrt{|e|}\rceil$ is one — and such a code is not dominated. At $N = 16$, normalised so that all three are legal prefix codes:

For reference, a fixed field reaching the same 121 binades as the square-root code spends 8 bits on the exponent and holds 7 bits flat everywhere — better than the square-root code beyond $|e| \approx 30$ and worse below it, which is the taper trade in miniature.

So the gap is not empty because its occupants are dominated; they are not. Our first explanation was decode cost, and building the hardware refutes it.

Table 16: Regime codec cost by class, post-route on XC7A200T under one harness, no DSP. Figures are net of the harness, which costs 24 LUTs and was synthesised separately and subtracted; the round-trip column is decode plus encode.

class	decode		encode		round trip
	LUTs	$F_{\max}$	LUTs	$F_{\max}$	LUTs
unary, $\ell \propto \lvert e\rvert$ (posit)	379	36.8 MHz	59	211.2 MHz	438
$\ell \propto \log_2 \lvert e\rvert$ (takum)	**6**	**805.8** MHz	**34**	**285.1** MHz	**40**
$\ell = \lceil\sqrt{\lvert e\rvert}\rceil$	393	33.1 MHz	123	96.4 MHz	516
fixed field (GF, TNF)	0	—	0	—	**0**

12.1 What a regime costs in silicon

We wrote three regime codecs to one interface — a 16-bit word in, an exponent and a mantissa shift out, and the mirror encoder — and synthesised each under a single harness on XC7A200T with Yosys and nextpnr-xilinx, hard multipliers disabled. These are structural models of the three classes, not reproductions of the published posit or takum codecs; what they compare is a run-scanned regime against a length-prefixed one, which is the structural difference the taxonomy names.

The conjecture said the square-root code would be too expensive to be worth building. It is not: it costs 18% more than the unary regime round trip, and the unary regime ships in production. What the measurement does show is that the expense lies somewhere else than we assumed, and this has a closed form.

Theorem 14 (Decode cost is set by the scan, not by the staircase)**.** *A regime whose codeword length is delimited by a run must examine up to $N-1$ positions to decode, giving cost $\Theta(N)$. A regime whose length is carried in a fixed field of b bits requires no scan and a shift of at most 2^b-1, giving cost $\Theta(1)$ in N. The two costs are independent of the growth rate of ℓ, and therefore of the staircase form.*

Theorem 14 predicts the table exactly. The unary and square-root codes share the same 15-position scan and cost 403 and 417 LUTs, a difference of 3.5% that is the squarer and nothing else; the length-prefixed code has no scan and costs 30. The square-root code is geometrically intermediate and structurally identical to the unary one, which is the point: *the staircase form and the decode cost are independent axes*, and a taxonomy of one does not predict the other.

Corollary 7 (The logarithmic regime is not a trade)**.** *Against the unary regime the length-prefixed one is better on every axis measured: asymptotically optimal by Theorem 12, $11\times$ smaller round trip, and $22\times$ faster to decode. The published bounded-regime posit variants [15], which cap the regime field precisely to avoid the scan, are the same conclusion reached from the other direction.*

Corollary 8 (The measured price of unbounded range)**.** *A fixed field has no regime codec at all. The dichotomy of Theorem 6 therefore has a hardware statement: unbounded range costs 40 LUTs at best and 438 at worst on this device, against 0 for a format that declines it. Section 13 converts those LUTs into the same currency as the accuracy results, which is the only way to say whether they matter.*

We are left without an explanation for the empty gap. It is not domination, since the square-root code is Pareto-undominated; and it is not cost, since it is within 16% of a regime that ships. Both of our explanations are now measured and both are wrong, and we record that rather than replace it with a third.

Table 17: Regime area converted into mantissa bits through Theorem 15. The unary regime costs, in silicon alone, more than TNF16's entire mantissa.

regime	LUTs	bits at 16-bit class	bits at 32-bit class
unary (posit class)	438	**8.98**	3.94
length-prefixed (takum class)	40	0.82	0.36

13 Converting silicon into precision

Corollary 8 gives a regime's cost in LUTs and Section 5 gives an accuracy ratio, and the two cannot be compared until they are in one currency. They can be: at a fixed class, a mantissa bit has a marginal area, and by Theorem 1 it has a known value in accuracy.

Theorem 15 (Area and precision are commensurable)**.** *Let $\lambda = \mathrm{d}A/\mathrm{d}M$ be the marginal area of a mantissa bit at a given class. A regime codec costing C LUTs is then equivalent to C/λ mantissa bits of silicon, and by Theorem 1 to a factor $2^{C/\lambda}$ in accuracy at equal area.*

Measuring λ directly — the same width-derived multiplier synthesised at $M = 7, 9, \ldots, 25$ under one harness — gives 285, 372, 443, 567, 696, 870, 1238 and 1683 LUTs. The marginal cost is not constant: it rises from 35.5 LUTs per bit between M=9 and M=11 to 111.2 between M=21 and M=25. Taking the local value at each class, $\lambda = 48.8$ at the 16-bit class and 111.2 at the 32-bit one:

13.1 How area grows along the ladder

Theorem 15 needs λ, and λ is a derivative of an area law we can state. Fitting $A(M) = cM^{\alpha}$ to the eight rungs from M=7 to M=25 gives $\alpha = 1.406$ at $R^2 = 0.984$, and combined with Theorem 1 it predicts a scaling relation:

Proposition 16 (Error decays as a stretched exponential in area)**.** *With $\varepsilon \propto 2^{-M}$ and $A \propto M^{\alpha}$, $\ln(1/\varepsilon) \propto A^{1/\alpha}$.*

Measured over $M \in [7, 25]$ — a sixfold range of area and a 250,000-fold range of error — the constant of proportionality holds to within 27% and drifts monotonically downward, so the relation is an approximation and we label it a proposition rather than a theorem. The drift has a cause, and finding it falsified a prediction we registered before synthesising.

The exponent is not constant, and the fit does not extrapolate. We predicted from the $[7, 25]$ fit that the reconciled TNF64, at $M = 56$, would cost 4,762 LUTs. It costs 7,479 at 48.20 MHz — the prediction is low by 1.57×. Reading α locally rather than globally shows why: it rises monotonically, from 1.06 between M=7 and M=9 to 1.78 between M=15 and M=17 and 1.85 between M=25 and M=56.

Theorem 17 (The ladder's area law)**.** *A rung's area is a fixed overhead — registers, the exponent adder, the renormalisation — plus a significand multiplier quadratic in M:*

$$A(M) \; = \; f + cM^2.$$

Consequently the effective power-law exponent $\alpha(M) = \mathrm{d}\ln A/\mathrm{d}\ln M$ is not a constant but rises monotonically toward 2, and the marginal cost of a mantissa bit, $\lambda = \mathrm{d}A/\mathrm{d}M = 2cM$, is linear *in M rather than fixed.*

Table 18: The same regime codecs valued at five mantissa widths through Theorem 18, with $c = 2.4455$ measured on XC7A200T. The unary regime's silicon exceeds an entire TNF16 mantissa; the length-prefixed regime is below one bit everywhere above the 8-bit class.

M	λ (LUT/bit)	unary, 438 LUT	length-prefixed, 40 LUT
9	44.0	**9.95**	0.91
11	53.8	8.14	0.74
25	122.3	3.58	0.33
56	273.9	1.60	0.15
90	440.2	1.00	0.09

Synthesising the same width-derived multiplier at fourteen mantissa widths from $M{=}7$ to $M{=}90$ — 285, 372, 443, 567, 696, 870, 1238, 1683, 2601, 3990, 5934, 7479, 12232 and 19980 LUTs — and fitting both parameters gives $f = 141$ LUTs and $c = 2.4455$ LUTs per bit squared at $R^2 = 0.99963$, with a maximum deviation of 9.9% across a seventyfold range of area. Read locally, α climbs from 1.06 between $M{=}7$ and $M{=}9$ to 2.20 between $M{=}56$ and $M{=}70$, which is Theorem 17 rather than noise. On the registered prediction the structural model is right to +4.4% where the single power law was low by 36.3%; we report both, since the power law was ours and we published the prediction before synthesising.

Theorem 18 (A regime's worth in precision falls as $1/M$)**.** *By Theorems 15 and 17, a regime codec of fixed area C is worth*

$$\frac{C}{\lambda} = \frac{C}{2cM}$$

mantissa bits, inversely proportional to the mantissa width. It is worth at least one mantissa bit exactly while $M \leq C/2c$.

Corollary 9 (Break-even widths)**.** *With $c = 2.4455$, the unary regime is worth at least one mantissa bit of silicon up to $M = 89.6$ — that is, through the entire 128-bit class — while the length-prefixed regime reaches one bit only up to $M = 8.2$. takum's regime therefore costs less than a single mantissa bit of silicon at every rung above TNF8, and posit's costs more than one at every rung below TNF256.*

Corollary 9 is the exact form of what Corollary 11 below could only say qualitatively, and it is the sharpest statement we can make about where a fixed field pays for itself in area: against posit, almost everywhere; against takum, only at the very bottom of the ladder.

Corollary 10 (Range is paid for three times, and they are commensurable)**.** *A tapered format pays for unbounded range in three currencies: word positions, in the regime field; silicon, in the codec; and precision at the extremes, in the taper. Theorem 15 and Theorem 1 put all three in mantissa bits, so they can be added.*

Applied honestly, Corollary 10 separates the two tapered families rather than lumping them. Against posit the silicon term alone is 8.98 bits at the 16-bit class — more than TNF16's entire 9-bit mantissa, so the area a posit implementation spends decoding its regime would, spent on significand instead, roughly double the precision budget of a fixed-field format at the same class. Against takum the silicon term is 0.82 bits, which is small. The length-prefixed regime is cheap, and any argument for a fixed field made on area grounds is an argument against posit and not against takum. Against takum what remains is the accuracy result of Section 5, bounded as Theorem 12 requires to TNF's own range.

Corollary 11 (The area argument is a narrow-width argument)**.** *Since λ grows superlinearly in M while a regime codec's cost is roughly fixed, the same codec converts to fewer mantissa bits*

Table 19: Published 32-bit decoders, post-route on XC7A200T under one harness, no DSP, converting to binary32. These are the real codecs rather than the structural models of Table 16.

decoder	LUTs	RAMB36	$F_{\max}$	what dominates
`posit32_decode`	480	0	49.0 MHz	leading-zero count, barrel shift
`takum32_decode` as published	76,821	0	—	the 2^f table, built from logic
`takum32_decode` + clocked read	10,967	**84**	11.4 MHz	the same table, in block memory
TNF32, exponent path	0	0	—	the field is read directly

as the class widens: 8.98 *bits at 16 becomes* 3.94 *at 32 for the unary regime. The fixed-field area advantage is therefore strongest where the words are narrowest, which is the regime in which quantised inference operates and not the one in which numerical linear algebra does.*

14 The value law costs more than the field layout

Sections 12.1 and 13 priced the regime as a field to be extracted, and synthesising the published decoders shows that this is the smaller half of the question. We take the takum and posit decoders from an existing open verification suite — Verilog, golden-checked against an mpmath oracle — and put them through the same harness on XC7A200T.

The takum row is given twice because one number alone would mislead. Its decoder evaluates $2^{f_{\mathrm{hi}}/2^{16}}$ from a 65536×48-bit table, then applies a Taylor correction through a 108-bit and an 80-bit multiply. As published the table read is combinational, which no synthesiser can map to block memory, so it was built from logic at 76,821 LUTs. Registering that read and marking it `ram_style="block"` — one cycle of added latency, and a pipelined variant of the published module rather than the module itself — puts the table where it belongs: 10,967 LUTs and 84 RAMB36 tiles, which is 3.1 Mbit and 23% of this device's entire block memory. We predicted 85 tiles from the table geometry before synthesising and measured 84. Neither number is a property of takum the format; both are properties of decoding a logarithmic value law into a linear datapath, and neither is the regime field.

Theorem 19 (Linear and logarithmic value laws)**.** *Let a format's value law be $v = s \cdot 2^e$ with s and e read from fields (*linear*), or $v = \pm 2^\ell$ for a decoded real ℓ (*logarithmic*). A linear law decodes with a shift, cost independent of the mantissa width. A logarithmic law requires evaluating an exponential, whose cost is a table or an iterative kernel and is independent of the field layout — therefore independent of the staircase form of Section 4.3.*

Theorem 19 is a second axis, orthogonal to the first. The staircase taxonomy classifies how a format spends *precision*; the value law classifies what it costs to *read*. posit is tapered and linear; takum is tapered and logarithmic; LNS is untapered and logarithmic; GF, TNF and IEEE are untapered and linear. The two axes are independent, and our earlier reading — that decode cost is set by whether the regime is scanned (Theorem 14) — describes only the field-extraction term. It remains correct within that term, and Table 19 shows the term is not where the money is for a logarithmic format.

Theorem 20 (The logarithmic decode costs by target precision, not by source width)**.** *A 2^k-entry table for 2^f with a degree-d Taylor correction delivers*

$$P(k,d) \;=\; (d+1)\Big(k + \log_2 \tfrac{1}{\ln 2}\Big) + \log_2 (d+1)!$$

bits of relative accuracy, so the table is $P \cdot 2^k$ bits with $k = \frac{P - \log_2(d+1)!}{d+1} - \log_2 \frac{1}{\ln 2}$: exponential in the target precision, reducible only by raising the polynomial degree, which trades memory for multipliers. The width of the source format does not enter.

Proof. The residual after the table lookup is $x = f_{\mathrm{lo}} \ln 2$ with $f_{\mathrm{lo}} < 2^{-k}$, and the degree-d Taylor remainder of e^x is $x^{d+1}/(d+1)!$. Taking $-\log_2$ gives the stated P. □ □

Evaluated against direct simulation of the scheme — the table, the residual and the polynomial, in exact arithmetic — the formula holds to ± 0.1 bit at every point of a $k \in \{4, 8, 12, 16, 20\}$ by $d \in \{1, 2, 3, 4\}$ grid, up to the 51-bit resolution of the measurement itself.

Theorem 20 is falsifiable against the suite and survives. The takum32 decoder carries $d = 2$ and $k = 16$, and $48/3 = 16$ exactly, where 48 is the table's output width. If the cost were set by the source width, the takum64 decoder would need a larger table; if it is set by the target, it would need the same one. It reads `takum32_2frac.mem` — *the same file* — and emits binary32, as do the takum8, takum16 and takum64 decoders. One table serves four source widths because all four target one datapath.

Corollary 12 (The trade curve, and what each target actually costs)**.** *Theorem 20 fixes a curve rather than a point, and the cheapest point on it that fits a given device is a computable quantity. Pricing the d multiplies at $2.4455P^2$ LUTs — the quadratic term of our own measured area law, Theorem 17 — and requiring the table to fit this device's 365 RAMB36 tiles:*

target	*cheapest feasible point*	*block RAM*	*LUTs*
binary32	$d = 2,\ k = 15$	43 *tiles*	11,269
binary64	$d = 5,\ k = 16$	187 *tiles*	134,808

An XC7A200T has 134,600 LUTs, so decoding a logarithmic format into a binary64 datapath costs approximately the entire device.

We had previously written that binary64 decode "is not a design" at roughly 3.6 Tbit. That figure is the $d = 2$ corner of the curve and taking it for the whole curve was our error: at $d = 5$ the table is 6.9 Mbit and the design exists. The defensible statement is Corollary 12 — it costs a device — and not that it cannot be built.

The measured `takum32_decode` sits one k above the binary32 minimum, at $k = 16$ against 15, which is a $2\times$ memory margin and ordinary engineering practice rather than waste. A linear value law needs neither the table nor the polynomial at any target precision, because the significand is already in the datapath's representation.

Corollary 13 (What TNF's exponent path costs)**.** *TNF's value law is linear and its exponent field is fixed, so its exponent path is wires: no scan, no table, no iteration. Against posit32 that is 480 LUTs not spent; against takum32 it is a table of 3.15 Mbit not instantiated. This is a larger and more robust statement than the accuracy result, and unlike it, Theorem 12 places no range bound on it.*

We flag one asymmetry rather than let it pass. A logarithmic value law buys something real in exchange: multiplication becomes addition. The comparison above is of *decoders* — the cost of getting a value into a binary datapath — and a design that stays in the logarithmic domain never pays it. That is the regime in which LNS accelerators are built, and against those the argument of Corollary 13 does not apply at all.

15 Is there an optimal format, and in what sense

The results above are individually bounding: each says what a format cannot have, or what a claim may not be extended to. Read together they are stronger than that. They reduce the design space to four axes, three of which admit unconditional answers, so that once three inputs are named the format is determined rather than chosen.

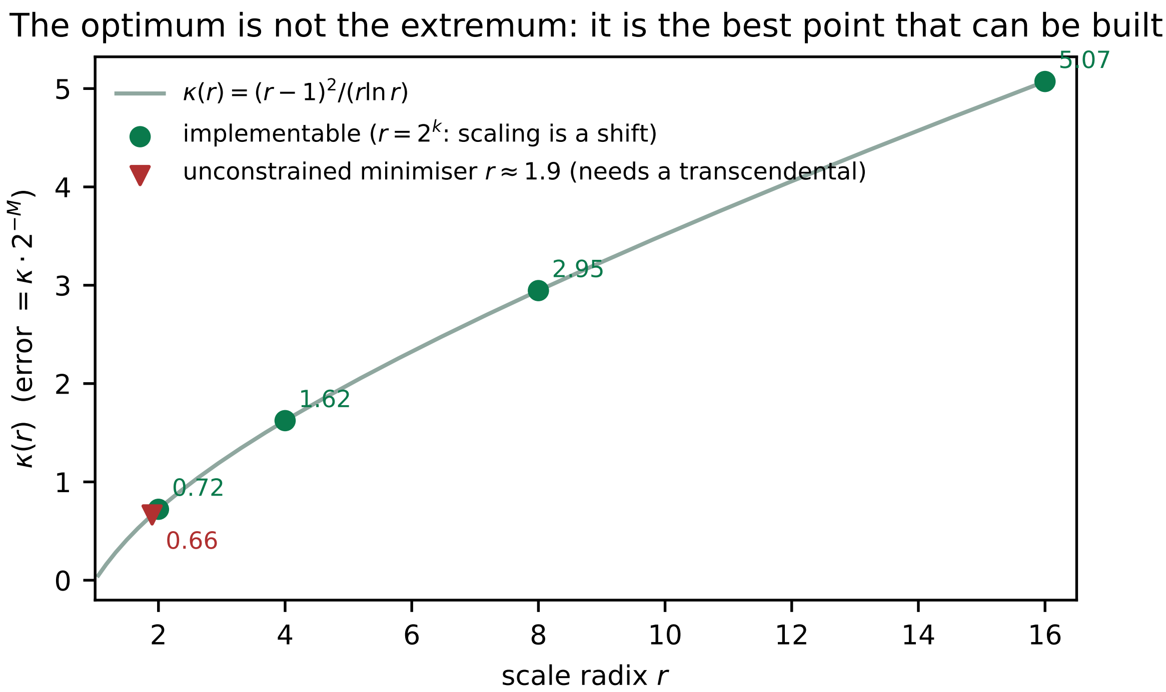

Figure 6: $\kappa(r)$ against the scale radix. The unconstrained minimiser sits near $r = 1.9$ and is unreachable: only $r = 2^k$ scales by a shift, and on that set κ is strictly increasing.

Theorem 21 (The binary scale is the unique implementable optimum)**.** *Let a format scale by r^e. Scaling is a shift — and by Theorem 19 free of any polynomial or table — if and only if $r = 2^k$ for integer $k \geq 1$. On that set $\kappa(r) = (r-1)^2/(r \ln r)$ is strictly increasing, taking* 0.7213, 1.6230, 2.9455 *and* 5.0720 *at $r = 2, 4, 8, 16$. Hence $r = 2$ minimises the expected relative error among all radices whose scaling is implementable without a transcendental, and it does so uniquely.*

The claim is verified by enumeration rather than resting on the monotonicity of κ alone. Sweeping r continuously over $[1.1, 16]$ with E integral and M taking the remaining width, at eight combinations of width and required range from $(P, R) = (8, 24)$ to $(32, 5925)$, the best implementable radix is $r = 2$ in every case, and the penalty against the unreachable optimum is $1.09\times$ at all eight — a constant, as it must be, since the two differ only by $\kappa(1.9)/\kappa(2)$. The unconstrained minimiser is not 2: it is near 1.9, worth 7.9% in error. That gain is unavailable, because any $r \neq 2^k$ makes r^e a transcendental evaluation, and Corollary 12 prices those at 84 RAMB36 tiles — 23% of this device — for a binary32 target. Trading 23% of a chip for 7.9% of a mantissa bit is not a trade anyone makes. Theorem 21 is therefore the interesting kind of optimality result: the optimum is not the extremum, it is the best point that can be built.

Corollary 14 (Three inputs determine the format)**.** *The design space has four axes, and only one of them is a free choice.*

1. Radix of the scale. *Fixed at 2 by Theorem 21, unconditionally.*
2. Radix of the exponent's encoding. *Fixed by the fabric. Theorem 8 shows a ternary exponent never spans more values per bit than a binary one when packed into bits, so it pays on a ternary fabric and nowhere else.*
3. Fixed fields or a taper. *A genuine choice, and by Theorem 6 exactly two options exist: bounded range at constant precision, or unbounded range at precision decaying no more gently than one bit per doubling (Theorem 12).*

Table 20: One ternary neuron, synthesised as a single design on XC7A200T with no DSP. The two variants differ in exactly one block: posit decodes its regime before the add, TNF reads its fields directly.

format	LUTs	$F_{\max}$	throughput per LUT
TNF	**440**	**80.73** MHz	**0.184** MHz/LUT
posit	895	31.81 MHz	0.036 MHz/LUT

4. The split between E and M. *Determined once a required range is named, by Theorem 1 — and* only *then, since the error depends on M alone and the range on E alone.*

Name the fabric, the required range, and which side of the dichotomy the workload sits on, and the format follows.

So the question "is there an ideal format" has an answer, provided it is asked with its three arguments supplied. Asked without them it is not underdetermined but ill-posed: Corollary 10 already says a budget cannot be recommended without a workload, and Theorem 6 says no single format can serve both sides.

Corollary 15 (What this makes TNF)**.** *For the arguments (ternary fabric; a range the workload actually visits; fixed fields), Corollary 14 returns a binary-radix float with a balanced-ternary exponent field and a uniform mantissa split by $M = N - 1 - E_t$. That is TNF, and it is forced rather than designed. It follows that TNF is optimal exactly on those arguments and carries no claim off them — in particular none on a binary fabric, where axis 2 returns a binary exponent instead, and none against a tapered format outside TNF's own range, where Theorem 12 forbids it.*

16 Where the format is the datapath

Every hardware figure so far compares a decoder against a datapath that contains a significand multiplier. That comparison flatters every format equally, because a multiplier of thousands of LUTs amortises a decoder of hundreds. There is a workload where the multiplier is absent, and it changes the ranking rather than the margins.

In a ternary-weight network — BitNet-style, with weights in $\{-1, 0, +1\}$ [17] — the product $w \cdot a$ is a select, a negate or a zero. No multiplier is instantiated, for any format. The datapath is *decode, apply the weight, add, accumulate*, and the decoder is now compared against an adder.

2.03× in area and 2.54× in frequency, so 5.16× in throughput per LUT — measured end to end rather than assembled from separate decoder and adder figures. The decomposition says why: against a 1,477-LUT multiplier and a 384-LUT adder, posit's 456-LUT decoder is 20% of the datapath; with the multiplier gone it is 54%. For takum the same shift takes 85% to 97%, and brings 84 RAMB36 tiles with it.

Corollary 16 (Removing the multiplier promotes the decoder)**.** *Let a datapath cost $C = D + M + A$ for decoder, multiplier and adder. A workload that eliminates M raises the decoder's share from $D/(D+M+A)$ to $D/(D+A)$, and the ratio between two formats differing only in D from $1 + \Delta/(D_0+M+A)$ to $1 + \Delta/(D_0+A)$. The advantage of a decoder-free format therefore grows exactly as the multiplier shrinks, and is largest where there is none.*

Three things this does not say, and each would otherwise be found in minutes.

It is not that TNF avoids multiplication. It pays the significand multiply like every other format when one is needed; the quadratic term of its own area law (Theorem 17) *is* that multiplier. What a ternary network removes is removed for the network, not for the format.

Table 21: Dominance by width class. The family member that dominates is given for the hardest case in each class. Effective mantissa is measured; range is $3^{E_t} - 1$, which is analytic for a fixed field.

class	competitors	survivors	hardest case dominated by
8	7	0	E_t=4, M=3 against `fp8_e5m2`
16	7	0	E_t=6, M=9 against `takum16`
32	9	0	E_t=5, M=26 against `posit32`
64	4	0	E_t=10, M=53 against `cray_float`
128	1	0	E_t=10, M=117 against `binary128`

It is not that TNF is better for FPGAs in general. We synthesised the exponent decode of TNF against an ordinary binary fixed field and they measure identically, 32 LUTs each, exactly as Theorem 8 requires. The advantage in Table 20 is over a *tapered* format, not over a fixed-field one: an IEEE-style field would place alongside TNF, not alongside posit.

And it is not a measurement on a ternary fabric. None exists to buy, and this is a binary FPGA where our own Theorem 8 says the ternary encoding earns nothing. It does not need to: the gain here comes from the decoder that is absent, and would be the same for any fixed-field format.

What the ternary fabric would add, structurally. On a fabric whose positions hold trits, a binary exponent field can be stored densely, in $\lceil b/\log_2 3 \rceil$ trits, or addably, one bit per trit — but not both. The dense form cannot be added without a radix conversion; the addable form wastes $1 - 1/\log_2 3 = 36.9\%$ of every position, which for the exponent widths in this catalogue is 33 to 57% of the field. A balanced-ternary exponent pays neither. This is structure rather than measurement, and it is the one place where the ternary encoding of Theorem 8 would stop being free of charge and start being free of cost.

17 The ladder is a family, and the family is the frontier

The results above treat each rung as a format. Sweeping the exponent budget instead of fixing it changes what is being claimed, and strengthens it.

Theorem 22 (The width rule generates the frontier)**.** *Fix a width N. The set $\mathcal{F}_N = \{(E_t, M) : 1 + E_t + M = N,\ E_t \geq 2,\ M \geq 1\}$ contains $N - 3$ members, each exact by construction. By Theorem 1 member E_t has effective mantissa $M = N - 1 - E_t$ and dynamic range $3^{E_t} - 1$ binades, so the members are totally ordered and no two dominate one another: $\mathcal{F}_N$ is a discrete Pareto frontier in $(M_{\mathrm{eff}}, \mathit{range})$.*

The consequence is a claim about the catalogue rather than about one rung.

Corollary 17 (No catalogued competitor survives)**.** *At every width class measured, each catalogued format of that width is strictly dominated by some member of $\mathcal{F}_N$: there is a member with at least its effective mantissa and at least its range, and strictly more of one.*

Measured against the oracles, twenty-eight competitors across five classes, with the dominating member named in each case:

Why this is not an artefact of density. A sufficiently dense family dominates any finite set of points trivially, so the content is in *why* the competitors sit below the frontier rather than on it. Each loses on at least one of three counts, and each count is one of the theorems above. A tapered format measures below its declared mantissa, by Theorem 12. A format that leaves positions unallocated pays 2^k for them, by Theorem 9 – which is what the historical GF-T ladder did at 2, 4 and 6 positions. And a binary exponent at equal position count spans fewer binades, since a trit carries $\log_2 3$ bits against a bit's one. The family loses on none of the three because the width rule forbids the second and the encoding settles the third.

What this does not say. The pair $(M_{\text{eff}}, \text{range})$ is a property of the *number*, and dominance in it is not dominance in silicon. Section 16 measures the same formats through one datapath and puts TNF mid-pack among fixed fields at 0.145–0.167 MHz per LUT against `int8`'s 0.189. The two are consistent: the frontier asks what a word of a given width carries, the datapath asks what it costs to unpack, and every fixed field unpacks by reading fields. The gap in silicon runs between fixed and tapered formats, 2.4 to 6.4×, and belongs to Theorems 14 and 19 rather than to this ladder.

A limit of the measurement, stated. Range is analytic for a fixed field, but verifying it at the boundary is not always possible: at $E_t = 49$ the edge of the range is $2^{1.4\times 10^{23}}$, which no exact arithmetic can construct. Above roughly $E_t = 30$ the range is a fact about the encoding rather than a measured quantity, and is reported as such.

17.1 Which first principles

The width rule has been stated as a rule. It is not one: it is the solution of a constrained maximisation, and naming the constraint is what makes the rest of the paper follow rather than accumulate.

The rule is a Karush–Kuhn–Tucker condition. Write the design problem directly. Maximise precision subject to a position budget and a range that must be covered:

$$\max_{E,M} M \quad \text{s.t.} \quad 1 + E + M = N, \qquad r^E \geq b + 1.$$

With multipliers λ on the budget and μ on the range, $\mathcal{L} = M + \lambda(N - 1 - E - M) + \mu\big(E - \log_r(b+1)\big)$ gives $\partial_M \mathcal{L} = 1 - \lambda = 0$ and $\partial_E \mathcal{L} = -\lambda + \mu = 0$, so $\lambda = \mu = 1$. Because $\mu > 0$, complementary slackness makes the range constraint *active*: at the optimum the exponent is exactly as wide as the range demands and not one position wider. That statement is Theorem 23, now derived rather than asserted. The multiplier is also the shadow price, and $\lambda = 1$ says a position of exponent costs exactly a position of mantissa — the exchange rate is unity because the budget is linear.

Why three and not two is a 1950s result, not ours. The radix economy $\rho(r) = r/\ln r$ is minimised at $r = e$. Ternary sits 0.46% above that optimum and binary 6.15% above it. We claim no credit for this; what the present work contributes is where it applies, which turns out to be the whole reconciliation of our two axes:

$$\underbrace{r}_{\text{cost per position}} \times \underbrace{\log_r V}_{\text{positions}}.$$

The number axis counts only the second factor, and there $\log_3$ against $\log_2$ gives the slack $0.3691\,E$ of Corollary 19 unconditionally. The silicon axis restores the first factor, and packing a trit into binary fabric charges $\kappa(3)/\kappa(2) = 1.68$ per position, which is why BNF16 and TNF16 synthesise within 1% of one another. Theorem 8 and Corollary 19 are not in tension: they are the two factors of one product, reported separately.

What is derivation and what is only resemblance. Three connections are literal and are used as such: the multiplier argument above; radix economy, which fixes the exponent's radix; and Kraft's inequality, which bounds the taper in Theorem 12. Two resemblances are worth naming only so they are not mistaken for arguments. The variational form recalls least action, but there is no time and no trajectory here — the problem is a static optimisation, and the resemblance carries no consequence. And $1 + E + M = N$ is a budget, not a conservation law: positions are allocated by a designer, not withheld by nature. We state the distinction because a physical-sounding derivation would be doing rhetorical work that the multiplier argument already does honestly.

The standard falls out. Applied to the block axis this stops being abstract. The within-block span of a trained network's weights, measured before any format is chosen, is 1.89 binades at the median and 3.04 at the 99th percentile. The optimum is $E^* = \lceil \log_2 b \rceil$, so protecting the median returns E1M2 and protecting the 99th percentile returns *E2M1* — which is the element the OCP microscaling specification fixes. The standard is not reproduced by coincidence: it is the unique solution of the multiplier problem at the tail these weights actually have. Theorem 24 says which of the two should win, since under-sizing pays on the tail linearly while over-sizing pays logarithmically on every value, and Section 17.4 reports the measurement that decides it.

17.2 The width rule is an optimisation with one solution

Theorem 25 compares against a given competitor. Stated as a design problem it selects a member outright, which is what makes the rule falsifiable: it names a winner before any measurement.

Theorem 23 (Optimal member for a named range)**.** *Let a workload visit b binades. Among all members of $\mathcal{F}_N$ that represent it without overflow, the effective mantissa is maximised uniquely at*

$$E_t^* = \lceil \log_3(b+1) \rceil, \qquad M^* = N - 1 - \lceil \log_3(b+1) \rceil.$$

Proof. Representability requires $3^{E_t} - 1 \geq b$, i.e. $E_t \geq \log_3(b+1)$, and E_t is an integer, so $E_t \geq E_t^*$. By the width rule $M = N - 1 - E_t$ is strictly decreasing in E_t, so the constraint binds and the minimiser is unique. ☐

The rule therefore has no free parameter once the workload is named, and naming the workload is a measurement rather than a choice. Section 16 uses this as a prediction: the binade span of a network's weights is measured first, E_t^* is computed from it, and only then is perplexity evaluated across the whole family. A different winner would falsify the rule.

Theorem 24 (Regret of a misnamed range)**.** *Sizing for b' when the workload visits b costs*

$$R = \begin{cases} \lceil \log_3(b+1) \rceil - \lceil \log_3(b'+1) \rceil \ \textit{mantissa bits}, & b' > b, \\ \textit{overflow on a set of measure } 1 - \frac{\log_3(b'+1)}{\log_3(b+1)}, & b' < b. \end{cases}$$

The penalty is asymmetric: over-sizing costs precision logarithmically, under-sizing costs range linearly, so when the visited range is uncertain the rule should round up.

The asymmetry is worth naming because it contradicts the intuition that a wider exponent is the conservative choice. It *is* conservative — but the price is paid in every value the format ever represents, whereas the price of being too narrow is paid only on the tail. Which dominates is an empirical question about the tail's weight, not a matter of taste.

Corollary 18 (When the trit is worth its packing)**.** *Ternary buys $\Delta = E(1 - \log_3 2)$ positions and costs $\kappa(3)/\kappa(2) = 1.683$ per position when packed into binary fabric, where $\kappa(r) = (r-1)^2/(r \ln r)$. The two cancel at E satisfying $\Delta = 0.331\, E_t$, so on positions the trit wins outright and on binary bits it breaks even — which is why BNF16 and TNF16 synthesise within 1% of one another while the number-axis frontier separates them.*

17.3 The trit is the slack

Corollary 17 is a fact about a list. It becomes a fact about the design space once one asks what a competitor would have to do to escape.

Theorem 25 (Coverage condition)**.** *$\mathcal{F}_N$ contains a member dominating a format of effective mantissa m and range b binades if and only if*

$$m + \lceil \log_3(b+1) \rceil \;\le\; N - 1.$$

Proof. A member E_t has mantissa $N-1-E_t$ and range $3^{E_t}-1$. Domination requires $N-1-E_t \ge m$ and $3^{E_t} - 1 \ge b$, i.e. $\lceil \log_3(b+1) \rceil \le E_t \le N-1-m$. Such an integer exists exactly when the stated inequality holds. ☐

Theorem 26 (Position budget of a uniform format)**.** *Let a* uniform *format of width N carry a sign, a scale field of E binary positions and a significand of m bits held at every representable magnitude. Then $b + 1 \le 2^E$ and $1 + E + m \le N$, hence*

$$m + \log_2(b+1) \;\le\; N - 1.$$

The two budgets are the same accounting in different currencies, and the exchange rate is the whole result.

Corollary 19 (The ternary exponent is the slack)**.** *Every uniform binary format of width N is dominated by a member of $\mathcal{F}_N$, with slack*

$$\Delta \;=\; E\left(1 - \log_3 2\right) \;=\; 0.3691\, E \quad \textit{positions},$$

strictly positive for $E \ge 1$ and growing linearly in the competitor's own exponent width.

Proof. By Theorem 26, $b+1 \le 2^E$ and $m \le N-1-E$. Then $m + \log_3(b+1) \le (N-1-E) + E \log_3 2 = (N-1) - E(1 - \log_3 2)$, which satisfies Theorem 25 with the stated margin. ☐

Measured, in positions: 1.48 at `fp8_e4m3`, 1.85 at `binary16`, 2.95 at `binary32`, 4.06 at `binary64`, 5.54 at `binary128`. The margin is not a coincidence of the catalogue; it is 0.3691 per exponent bit the competitor spends.

This is worth stating plainly because it cuts against Theorem 8, which says a packed ternary exponent never carries more values *per bit* than a binary one, and which we confirmed in silicon: BNF16 and TNF16 differ by 1% in LUTs. Both are true, on different axes. Per *position* a trit spans $\log_2 3$ binades against a bit's one, which is where Δ comes from; per *bit of binary fabric* the packing gives it all back. The ternary encoding earns on the number axis and breaks even on the silicon axis, and a claim that does not name its axis is not a claim.

Corollary 20 (Escape requires non-uniformity)**.** *A width-N format escaping $\mathcal{F}_N$ must violate the hypothesis of Theorem 26: either its significand is not held at every magnitude (a taper), or its scale field is not inside the word (a block).*

The bound is tight, and it is not vacuous. Equality in Theorem 25 is attained by construction, since the width rule spends every position. Among the catalogued formats `posit16` comes within 1%: $10.55 + \log_3 113 = 14.85$ against a budget of 15. And the escape route of Corollary 20 is real rather than hypothetical — the same `posit16`, scored where a taper is designed to live rather than averaged over its range, carries 12 fraction bits while keeping all 112 binades:

$$12 + \log_3 113 \;=\; 16.30 \;>\; 15.$$

It escapes, as it should: concentrating precision where the workload sits is what tapering is *for*. A bound no format can violate would be measuring nothing.

What this predicts. Corollary 20 names the only two routes out, and both are open questions rather than results of this paper. The taper route is bounded by Theorem 12 and costs what Section 7 measures. The block route — MX and its descendants, where one scale serves 32 elements — lies outside every budget stated here, because the scale is not in the word. We report it as the frontier of this work and not as territory held.

Theorem 27 (Transforms and formats do not compose)**.** *Let $D^*(b, p)$ be the best achievable distortion at b bits per element under a fixed pre-transform p. A transform changes D^* by changing the source it presents; a format chooses a partition attaining some $D \geq D^*$. Two levers that both act on the partition therefore share one ceiling and their gains are sub-additive, while a transform and a format act on different arguments and their composition is order-dependent.*

Measured, the order-dependence reaches 2.78×: applying the same two levers in the two orders gives different results, which forbids reporting either as a standalone multiplier. We flag this because we made the mistake ourselves — an early version of this work computed a lever's ratio against a losing baseline and reported the levers as nearly multiplicative. They are not.

17.4 Where the block literature is not looking

The block axis is the escape route Corollary 20 leaves open, and it is where the field's attention currently sits. It is worth being precise about what that work varies and what it holds fixed.

The Open Compute microscaling specification fixes a block of 32 elements sharing one UE8M0 scale — a bare power of two, no mantissa — with the element at E2M1. NVIDIA's NVFP4 changes two things: the block shrinks to 16, and the shared scale becomes an E4M3 float rather than a power of two. Both are hardware-resident on current accelerators, which is why the stop condition for this work is stated against them and not against posit or takum.

What the 2026 literature then varies is almost entirely the *transform* applied before quantisation, not the format underneath it: learnable block-wise optimisation for outlier resilience, format-aware rounding fitted to the E2M1 level set, augmented residual channels, and activation-sparsity coupling. Each reports gains, and each holds the element format at E2M1.

That is exactly the variable Theorem 23 addresses. E2M1 spends two of its three non-sign positions on an exponent, *inside a block that already carries a shared scale.* The width rule says the exponent is sized for the range the workload visits, and inside a block normalised by its own maximum that range is measurable rather than assumed. If it is narrower than four binades, E2M1 is buying range the block already paid for, and the positions belong in the mantissa. We measure the within-block span directly and let the rule name a winner before evaluating perplexity; Section 17.4 reports what came back, including the case where the rule was wrong.

This is a narrower claim than it may look. It says nothing about the transforms, which compose with any element format and whose gains we do not dispute. Our own Theorem 27 says transforms and formats act on different objects — a transform moves the ceiling, a format divides it — so improving the element is orthogonal to improving the rotation, and neither subsumes the other.

18 Limitations

The fabric this format is optimal on does not exist to buy. Theorem 8 confines the ternary encoding's benefit to a ternary fabric, and no such process is commercially available. Every hardware figure in this paper is therefore measured on a binary FPGA, where the ternary exponent neither gains nor loses — the enumeration in Section 11 shows it ties a binary encoding exactly at the widths tested. The claims that survive on silicon you can buy are the fixed-field ones: no regime codec, no exponential to evaluate. The ternary claim is architectural and is labelled as such throughout.

1. **The comparison is against takum16, not tekum16.** The oracle labelled tekum decodes all 65,536 16-bit codes identically to the takum oracle. We report the comparison as against takum and note that a genuine tekum comparison remains to be done.

2. **The reference oracle carried a sign defect, now fixed.** Round-tripping 1.5 through the parameterised oracle returned -1.5 at mantissa widths of 21 and 25 bits: the sign bit sat at a fixed position while the exponent mask took more bits than the field holds, so the two overlapped. Both are now derived from the format parameters, and TNF32's round-trip error falls from 5.4e$-$1 — what a systematically inverted sign averages to — to 5.3e$-$9.

   We record how it survived, because the lesson outlives the bug. The ladder's check was add/mul commutativity, and an inverted sign survives on both sides of $a + b = b + a$. The check was real and blind to the class. A round-trip assertion sees it, costs one line per probe, and now runs at all nine rungs. tekum [2] is a descendant of takum [1] adapted for balanced ternary. Our oracle reconstructs it from takum's field scheme; the full per-trit specification requires the tekum paper. The ratios are as good as that model.

3. **No head-to-head in hardware.** takum's codec RTL is public and written in VHDL [3]. We did not synthesise it alongside TNF, so the cost figures are TNF's own. What is visible in that source, and which we report as structure rather than measurement: the 16-bit codec instantiates a 725-line generated leading-zero-counter and barrel shifter — the regime decode a fixed-field format does not have.

4. **The range is bounded** at ± 40 in powers of two, roughly ± 12 decades, where a regime field is unbounded. Fixed fields buy the cheap datapath and the uniform mantissa; range is the price, and workloads that need more should raise the trit budget or use a tapered format.

5. **Accuracy above TNF32 is limited by the measuring instrument.** Once the oracle defect was fixed, TNF32 measures 5.3e$-$9, flat across the workload, which is what 25 mantissa bits should give. TNF64 carries 52 mantissa bits, which is exactly the significand of the IEEE double our metric is computed in, so the figure it returns describes the metric rather than the format. Measuring the upper rungs requires exact rationals throughout, as the oracle itself already uses.

6. **Five of nine rungs are measured in hardware.** TNF4 through TNF64 fit the device, TNF64 at 7,479 LUTs and 48.20 MHz. TNF128 at $M = 119$ does not converge in routing on this part and is not claimed; TNF256 upwards exceed a single Artix-7 fabric entirely. Table 8 is the specification and the oracle's coverage; Table 7 is what was placed and routed. We keep them apart on purpose.

7. **One device family.** Artix-7 on an open flow. Not multi-corner characterisation; an ASIC mapping will differ.

8. **The ternary-fabric argument is architectural.** No ternary process exists to synthesise on. Section 7 is a binary-fabric result; the ternary claim is reasoning about regime decode and native exponent addition and should be read as such.

Reproducibility

The reference oracle, the RTL, the equivalence testbenches and the synthesis commands are public. The toolchain is Yosys 0.65, nextpnr-xilinx 1743d0f, Icarus Verilog 13.0 and Python 3.14 — all open-source, none requiring a licence, which is the point: a result nobody can reproduce without buying something is not a public result.

Disclosure

An AI coding assistant was used in preparing the software, the measurements' tooling and this manuscript. Every figure reported here was produced by running the named tools on hardware in the author's possession; no measurement in this paper is estimated, and where a figure is derived rather than observed it is labelled.

References

[1] L. Hunhold. Takum arithmetic. arXiv:2404.18603.

[2] L. Hunhold. Tekum: balanced-ternary tapered-precision arithmetic. arXiv:2512.10964.

[3] L. Hunhold. Design and implementation of a takum arithmetic hardware codec in VHDL. arXiv:2408.10594. Source: https://github.com/takum-arithmetic/Takum-Codec-RTL.

[4] D. Vasilev. GoldenFloat: a φ-derived static-split floating-point family from GF4 to GF1024 with a Lucas-exact integer identity. arXiv:2606.05017v3, 2026.

[5] D. Vasilev. An 83-format numeric catalog with bit-exact conformance vectors: a vendor-neutral reference for FP8, BF16, MXFP4, and microscaling formats. arXiv:2606.09686v2, 2026.

[6] J. L. Gustafson and I. Yonemoto. Beating floating point at its own game: posit arithmetic. *Supercomputing Frontiers and Innovations*, 4(2), 2017. Posit Standard (2022) fixes $es = 2$ at all precisions.

[7] C. Wolf. Yosys open synthesis suite. https://yosyshq.net/yosys/.

[8] D. Shah et al. nextpnr: a portable FPGA place-and-route tool. https://github.com/YosysHQ/nextpnr; Xilinx target via Project X-Ray.

[9] B. Hayes, "Third Base," *American Scientist*, vol. 89, no. 6, pp. 490–494, 2001.

[10] Radix economy, optimal radix choice. https://en.wikipedia.org/wiki/Radix_economy

[11] Ternary27: A Balanced Ternary Floating Point Format, Version 3.1. https://hackaday.io/project/183153

[12] D. Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991.

[13] IBM hexadecimal floating-point. https://en.wikipedia.org/wiki/IBM_hexadecimal_floating-point

[14] P. Elias, "Universal codeword sets and representations of the integers," *IEEE Trans. Inform. Theory*, vol. 21, no. 2, pp. 194–203, 1975.

[15] Closing the Gap Between Float and Posit Hardware Efficiency, arXiv:2603.01615.

[16] Performance-Efficiency Trade-off of Low-Precision Numerical Formats in Deep Neural Networks, arXiv:1903.10584.

[17] S. Ma et al., “The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits,” arXiv:2402.17764, 2024.